

# AeroNet Lite

*Autonomous Drone Delivery Simulation with CSP, Fleet Planning, A\* Routing, Real-Time Replanning, Demand Forecasting, and Anomaly Detection*

## Document Purpose

This guide explains exactly what to build, how to build it, which algorithms to use, which datasets are valid, and how to demonstrate the system within a 2-week deadline.

| Original Scope Problem                                                               | Simplified Decision                                                                 |
|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Full city-scale autonomous drone fleet                                               | 10x10 simulation grid                                                               |
| Five heavy modules with ACO, GA, CSP, A*, clustering, regression, and classification | Five lighter modules using CSP, heuristic or GA, A*, regression, and classification |
| Complex vehicle routing problem                                                      | Nearest-drone assignment plus A* path planning                                      |
| Full real-time UI with multiple overlays                                             | Simple visual dashboard using matplotlib or Streamlit                               |
| Large integrated simulator                                                           | 20-step console and visual simulation                                               |

## 1. Final Project Scope

AeroNet Lite is a drone delivery simulator. It models a city as a 10x10 grid. Each grid cell has a zone type, population density, facility flags, and a no-fly status. The system validates the layout, chooses a small fleet, plans routes, reacts to disruptions, predicts demand, and detects flight anomalies.

| Module                    | What it does                                            | AI technique                          |
|---------------------------|---------------------------------------------------------|---------------------------------------|
| Grid and Layout Validator | Checks city layout rules and reports violations         | Constraint Satisfaction Problem       |
| Fleet Selector            | Selects number of light and heavy drones under a budget | Heuristic search or Genetic Algorithm |
| Delivery Path Planner     | Finds shortest safe route for deliveries                | A* Search                             |
| Disruption Handler        | Activates no-fly cells and reroutes affected drones     | Optimization Algorithm                |
| ML Pipeline               | Predicts demand and detects flight anomalies            | Regression and Classification         |

| Component          | Reads from shared model               | Writes to shared model                    |
|--------------------|---------------------------------------|-------------------------------------------|
| Layout Validator   | Zone type, hub flags, charging flags  | Validation results only                   |
| Fleet Selector     | Hub locations and demand estimates    | Selected drone list                       |
| Path Planner       | No-fly flags, costs, hubs, deliveries | Routes                                    |
| Disruption Handler | Current routes and no-fly flags       | Updated no-fly flags and reroute requests |
| ML Pipeline        | Demand history and flight logs        | Forecasts and anomaly alerts              |
| UI and Event Log   | Everything                            | Nothing except user-triggered events      |

## 2. Shared Grid Model

Use a 10x10 grid. Each cell can be represented as a Python dictionary or a dataclass. A dictionary is easier for beginners; a dataclass is cleaner for a professional implementation.

## Sample 10x10 Region Grid



## Recommended Cell Structure

| Field             | Example       | Purpose                                      |
|-------------------|---------------|----------------------------------------------|
| row, col          | (3, 7)        | Stores cell position                         |
| zone              | Residential   | Defines the type of area                     |
| density           | 5000          | Used for demand estimation and visualization |
| is_hub            | True / False  | Marks drone hub locations                    |
| is_charging       | True / False  | Marks charging pad locations                 |
| is_medical_pickup | True / False  | Marks medical supply pickup points           |
| no_fly            | True / False  | Blocks routing through this cell             |
| demand            | numeric value | Estimated delivery demand                    |

### 3. Module 1 - Layout Validation Using CSP

This module checks whether the grid satisfies required operating rules. Students do not need to automatically generate the entire city. They can manually define the grid, then run a validator that behaves like a CSP constraint checker.

| Rule ID | Constraint                                                             | Implementation method                                                 |
|---------|------------------------------------------------------------------------|-----------------------------------------------------------------------|
| R1      | Industrial cells cannot be directly adjacent to Schools or Hospitals   | For each industrial cell, check four neighbors: up, down, left, right |
| R2      | Every Residential cell must be within 3 Manhattan cells of a Drone Hub | For every residential cell, compute Manhattan distance to each hub    |
| R3      | Every Drone Hub must have a Charging Pad within 2 cells                | For every hub, compute Manhattan distance to every charging pad       |
| R4      | At least one Hospital must have a Medical Pickup point within 1 cell   | Check all hospital cells against all medical pickup cells             |

#### Step-by-step implementation

- Create a function `get_neighbors(row, col)` that returns valid adjacent cells.
- Create a function `manhattan(a, b)` that returns  $\text{abs}(r1-r2) + \text{abs}(c1-c2)$ .
- Create one validation function per rule: `check_industrial_safety`, `check_residential_coverage`, `check_hub_charging`, `check_medical_access`.
- Store errors in a list instead of stopping after the first error.
- Print a professional report: Passed rules, failed rules, and cell coordinates causing the problem.

#### Expected output

Example: Layout validity = False. Failed rule R2: Residential cell (7, 5) is more than 3 cells away from every hub.  
Suggested fix: add a hub near (6, 5) or convert that cell to Open Field.

### 4. Module 2 - Fleet Selection

This module chooses a small drone fleet under a fixed budget. To keep the work manageable, use only two drone types. A brute-force search is acceptable. A small Genetic Algorithm can be used for stronger AI coverage.

| Drone Type  | Cost | Payload | Range    | Use case                              |
|-------------|------|---------|----------|---------------------------------------|
| Light Drone | 1000 | 2 kg    | 12 cells | Small parcels and nearby deliveries   |
| Heavy Drone | 1800 | 5 kg    | 20 cells | Medical packages or longer deliveries |

#### Simplified fitness function

Use a score that rewards demand coverage and penalizes cost. A simple formula is:

$$\text{score} = (0.75 \times \text{coverage\_percentage}) - (0.25 \times \text{budget\_used\_percentage})$$

A good solution serves most demand without spending the full budget unnecessarily.

## Implementation options

| Option            | How to implement                                                                  |
|-------------------|-----------------------------------------------------------------------------------|
| Brute force       | Try all combinations from 0 to max drones and keep the highest score              |
| Genetic Algorithm | Chromosome = [light_count, heavy_count]. Apply mutation, crossover, and selection |

## 5. Module 3 - Delivery Path Planning with A\*

This module plans safe routes through the grid. A delivery route is: hub -> pickup -> drop-off -> hub. Each segment is solved using A\* search.

### A\* design choices

| Item          | Choice                                           | Reason                                   |
|---------------|--------------------------------------------------|------------------------------------------|
| State         | Grid coordinate (row, col)                       | Simple and easy to visualize             |
| Actions       | Move up, down, left, right                       | Matches grid adjacency                   |
| Path cost     | 1.0 per normal move, 0.8 for commercial corridor | Matches operational model                |
| Blocked nodes | Cells where no_fly = True                        | Safety rule                              |
| Heuristic     | Manhattan distance                               | Admissible for 4-direction grid movement |

### Implementation steps

- Create function astar(start, goal, grid).
- Use a priority queue where priority = g\_cost + heuristic.
- Skip neighbors where no\_fly is True.
- Store parent pointers to reconstruct the path.
- Return path, total cost, and a failure message if no path exists.

## 6. Module 4 - Real-Time Disruption Handling

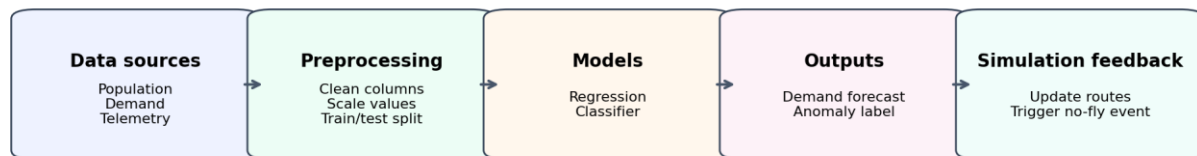
At any simulation step, a cell can become no-fly. If a drone route contains that cell, the system must rerun A\* from the drone current location to its next target.

| Event                     | System action                                                | Event log example                        |
|---------------------------|--------------------------------------------------------------|------------------------------------------|
| No-fly cell activated     | Update grid[row][col].no_fly = True                          | Step 6: no-fly cell activated at (4, 7)  |
| Route crosses no-fly cell | Call A* again from current drone position                    | Drone D2 rerouted using A*               |
| No safe route exists      | Mark delivery as delayed or failed                           | Drone D3 cannot reach destination safely |
| Anomaly detected          | Optional: activate nearby no-fly cell or force return to hub | Critical anomaly detected for D1         |

## 7. Module 5 - Demand Forecasting and Anomaly Detection

This module provides the machine learning part of the project. It should remain simple, explainable, and easy to defend during viva.

### Machine Learning Pipeline Used in AeroNet Lite



### Demand forecasting

| Step | Action                                                                          |
|------|---------------------------------------------------------------------------------|
| 1    | Load the Bike Sharing Demand dataset or Amazon Delivery dataset.                |
| 2    | Select simple features such as hour, day, temperature, weather, or zone demand. |
| 3    | Split data into training and testing sets.                                      |
| 4    | Train Linear Regression or Random Forest Regressor.                             |
| 5    | Report MAE or RMSE.                                                             |
| 6    | Use predicted demand to influence fleet selection and delivery generation.      |

### Anomaly detection / classification

For a 2-week project, synthetic anomaly labels are acceptable if the logic is clearly explained.

| Class           | Synthetic rule                                         | Example features affected            |
|-----------------|--------------------------------------------------------|--------------------------------------|
| Normal          | Battery decreases gradually and route deviation is low | battery_drop, speed, deviation       |
| Battery anomaly | Battery suddenly drops faster than expected            | battery_drop high                    |
| Route anomaly   | Drone moves far from planned path                      | route_deviation high                 |
| Sensor spike    | Altitude or speed suddenly jumps                       | altitude_change or speed_change high |

Train a Decision Tree or Random Forest classifier. Report accuracy and confusion matrix. If time allows, compare with KNN or Naive Bayes for stronger evaluation.

## 8. Valid Data Resources

The project can combine real public datasets with synthetic data. This is acceptable because the main goal is to demonstrate AI techniques and integration, not to deploy a real drone system.

| Purpose                     | Recommended source                                                                                                                                                                                                                                                                   | How to use it                                                                 |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Population density          | Kaggle - US City Population Densities: <a href="https://www.kaggle.com/datasets/mmcgurr/us-city-population-densities">https://www.kaggle.com/datasets/mmcgurr/us-city-population-densities</a>                                                                                       | Map density values into Low, Medium, High grid cells                          |
| Demand forecasting          | Kaggle - Bike Sharing Demand: <a href="https://www.kaggle.com/c/bike-sharing-demand">https://www.kaggle.com/c/bike-sharing-demand</a>                                                                                                                                                | Use hourly demand as a proxy for delivery demand                              |
| Delivery demand alternative | Kaggle - Amazon Delivery Dataset: <a href="https://www.kaggle.com/datasets/sujalsuthar/amazon-delivery-dataset">https://www.kaggle.com/datasets/sujalsuthar/amazon-delivery-dataset</a>                                                                                              | Use order and delivery features for demand or delivery-time prediction        |
| Drone telemetry / faults    | CMU KiltHub - ALFA UAV Fault and Anomaly Detection: <a href="https://kiltHub.cmu.edu/articles/dataset/ALFA_A_Dataset_for_UAV_Fault_and_Anomaly_Detection/12707963">https://kiltHub.cmu.edu/articles/dataset/ALFA_A_Dataset_for_UAV_Fault_and_Anomaly_Detection/12707963</a>          | Use UAV fault data for anomaly classification if students want real telemetry |
| Drone telemetry alternative | Kaggle - Supplemental Drone Telemetry Data and Operations Log: <a href="https://www.kaggle.com/datasets/samsudeenashad/supplemental-drone-telemetry-data-and-operations-log">https://www.kaggle.com/datasets/samsudeenashad/supplemental-drone-telemetry-data-and-operations-log</a> | Use flight logs for anomaly or operational event analysis                     |

### Recommended practical approach

- Use Bike Sharing Demand for regression because it is clean and beginner-friendly.
- Use synthetic anomaly data unless real UAV telemetry is easy to load.
- Use population density only to assign grid density levels, not for complex geographic mapping.
- Clearly mention all assumptions in the report.

## 9. 20-Step Simulation Scenario

The final demo should run a small simulation for 20 steps. Each step updates drones, checks for events, and prints logs.

| Step range | What happens                                   |
|------------|------------------------------------------------|
| 1-3        | Initialize grid, validate layout, select fleet |
| 4-6        | Generate deliveries and compute routes         |
| 7-10       | Move drones along planned paths                |
| 11         | Activate a no-fly cell                         |
| 12-14      | Reroute affected drones                        |

|       |                                                            |
|-------|------------------------------------------------------------|
| 15-17 | Run demand forecast and optionally add one delivery        |
| 18    | Inject or detect an anomaly                                |
| 19    | Reroute or force return to hub if needed                   |
| 20    | Print final summary: completed, delayed, failed deliveries |

### Minimum event log examples

- Step 1: Layout validation passed.
- Step 3: Fleet selected: 3 light drones, 2 heavy drones.
- Step 5: Delivery 4 assigned to Drone D2.
- Step 11: No-fly cell activated at (4, 7).
- Step 12: Drone D2 rerouted successfully.
- Step 18: Battery anomaly detected for Drone D3.
- Step 20: Simulation complete. 8 completed, 1 delayed, 1 failed.

## 10. Interface Requirements

A simple visual interface is enough. Students may use matplotlib for static updates or Streamlit for an interactive dashboard.

| View           | What to show                                                      | Minimum implementation      |
|----------------|-------------------------------------------------------------------|-----------------------------|
| Zone map       | Residential, commercial, hospital, school, industrial, open field | Colored grid                |
| Route map      | Drone paths and no-fly cells                                      | Colored grid with path line |
| Demand heatmap | Demand intensity by cell                                          | matplotlib heatmap          |
| Anomaly view   | Drone events and anomaly labels                                   | Table or printed log        |
| Event log      | Simulation decisions step by step                                 | Console text or UI panel    |

| Day | Task                                           | Deliverable                      |
|-----|------------------------------------------------|----------------------------------|
| 1   | Set up project folders and shared grid model   | grid.py and sample grid          |
| 2   | Add visualization for zone map                 | First grid plot                  |
| 3   | Implement CSP validation rules R1 and R2       | Validation report                |
| 4   | Implement CSP rules R3 and R4 plus suggestions | Complete layout validator        |
| 5   | Implement drone types and fleet scoring        | Fleet selection output           |
| 6   | Test heuristic or GA fleet selector            | Selected fleet under budget      |
| 7   | Implement A* path planner                      | Shortest path between two cells  |
| 8   | Add pickup/drop-off routing                    | Complete route for each delivery |
| 9   | Generate 5-10 deliveries and assign drones     | Delivery assignment table        |
| 10  | Implement no-fly disruption and rerouting      | Reroute event log                |
| 11  | Load demand dataset and train regression model | MAE or RMSE result               |
| 12  | Create or load anomaly dataset                 | Prepared anomaly labels          |
| 13  | Train classifier and print confusion matrix    | Classifier result                |
| 14  | Integrate all modules and rehearse demo        | 20-step simulation               |

## 11. Recommended Project Folder Structure

Use a simple structure so every group member can understand the project during viva.

```
aeronet_lite/
  data/
    raw/
    processed/
  src/
    grid_model.py
    layout_validator.py
    fleet_selector.py
    astar_planner.py
    delivery_simulator.py
    ml_pipeline.py
    visualization.py
    main.py
  notebooks/
    demand_forecasting.ipynb
    anomaly_classifier.ipynb
  report/
    figures/
    final_report.docx
  README.md
```

## 12. Rubric Alignment

| Rubric area                | How AeroNet Lite satisfies it                                                          |
|----------------------------|----------------------------------------------------------------------------------------|
| Design document            | Explains each module, algorithm choice, data model, UI plan, and team responsibilities |
| Technical implementation   | Each module works separately and integrates through shared grid model                  |
| AI concept coverage        | CSP, search, optimization, regression, and classification are included                 |
| Viva and defense           | Algorithms are simple enough for students to explain from first principles             |
| Interface and presentation | Grid visualization, route overlay, heatmap, and event log are included                 |

## 13. Final Submission Checklist

- Working Python project with clear folder structure.
- 10x10 grid visualization.
- CSP layout validator with failed rule reporting.
- Fleet selection result under budget.
- A\* route planner that avoids no-fly cells.
- Rerouting demonstration after disruption.
- Regression model with error metric.
- Classifier with accuracy and confusion matrix.
- 20-step simulation event log.
- Short final report.